

NAVIGATION			
Action	MCP Tool	CLI Skill	Test Runner
Open / Navigate	browser_navigate	open <url>	npx playwright test
Go Back	browser_navigate_back	go-back	...
Go Forward	browser_navigate_forward	go-forward	...
Reload	...	reload	...
Close Browser	browser_close	close	...

INTERACTIONS			
Action	MCP Tool	CLI Skill	Test Runner
Click	browser_click	click <ref>	...
Double Click	...	dblclick <ref>	...
Type Text	browser_type	type <text>	...
Fill Input	...	fill <ref> <text>	...
Hover	browser_hover	hover <ref>	...
Drag & Drop	browser_drag	drag <s> <e>	...
Select Option	browser_select_option	select <ref> <v>	...
Check / Uncheck	...	check uncheck <ref>	...
Press Key	browser_press_key	press <key>	...

CAPTURE			
Action	MCP Tool	CLI Skill	Test Runner
Snapshot (a11y)	browser_snapshot	snapshot	...
Screenshot	browser_take_screenshot	screenshot [ref]	--screenshot
Save as PDF	browser_pdf_save	pdf	npx playwright pdf
Resize	browser_resize	resize <w> <h>	--viewport-size

TABS			
Action	MCP Tool	CLI Skill	Test Runner
List Tabs	browser_tab_list	tab-list	...
New Tab	browser_tab_new	tab-new [url]	...
Select Tab	browser_tab_select	tab-select <i>	...
Close Tab	browser_tab_close	tab-close [i]	...

PLAYWRIGHT COMMAND REFERENCE

@playwright/mcp | @playwright/cli | @playwright/test | 50+ commands across all categories

DEVTOOLS			
Action	MCP Tool	CLI Skill	Test Runner
Console Msgs	browser_console_messages	console [level]	...
Network Reqs	browser_network_requests	network	...
Dialog	browser_handle_dialog	dialog-accept dismiss	...
File Upload	browser_file_upload	upload <file>	...
Eval JS	...	eval <fn> [ref]	...
Run Code	...	run-code <code>	...

TESTING & CODEGEN			
Action	MCP Tool	CLI Skill	Test Runner
Generate Test	browser_generate_playwright_test	...	npx playwright codegen
Run Tests	...	...	npx playwright test
Run Headed	...	open --headed	npx playwright test --headed
Debug	...	...	npx playwright test --debug
UI Mode	...	...	npx playwright test --ui
Show Report	...	...	npx playwright show-report
Show Trace	...	tracing-start stop	npx playwright show-trace
Wait For	browser_wait_for	...	...
Install	browser_install	...	npx playwright install

SESSION & MOUSE (CLI only)			
Action	MCP Tool	CLI Skill	Test Runner
List Sessions	...	session-list	...
Stop Session	...	session-stop [n]	...
Stop All	...	session-stop-all	...
Delete Session	...	session-delete [n]	...
Mouse Move	...	mousemove <x> <y>	...
Mouse Btn	...	mousedown mouseup	...
Mouse Wheel	...	mousewheel <dx> <dy>	...
Keydown/up	...	keydown keyup <key>	...

MCP SERVER@playwright/mcp

WHAT IT IS

A Model Context Protocol server that exposes browser automation tools to AI agents. Uses structured accessibility snapshots instead of screenshots.

INSTALL

```
npx @playwright/mcp@latest
```

CONFIGURATION

```
{  "mcpServers": {    "playwright": {      "command": "npx",      "args": [        "@playwright/mcp@latest"      ]    }  }}
```

KEY FLAGS

```
--browser chrome|firefox|webkit
--headless      (default: headed)
--caps vision   (enable screenshots)
--device 'iPhone 15'
--isolated      (fresh profile)
--save-trace    (record trace)
```

SUPPORTED CLIENTS

Claude Desktop | Cursor | VS Code
Windsurf | Goose | Claude Code
Amp | Factory | LM Studio

CLI SKILLS@playwright/cli

WHAT IT IS

A token-efficient command-line interface for coding agents. Avoids loading large tool schemas into the AI context window. Uses concise shell commands instead.

INSTALL

```
npm install -g @playwright/cli@latest
```

VERIFY & START

```
$ playwright-cli --help
$ playwright-cli open <url> --headed
$ playwright-cli snapshot
$ playwright-cli click <ref>
$ playwright-cli type 'Hello world'
$ playwright-cli press Enter
$ playwright-cli screenshot
```

SESSIONS (persistent state)

```
$ playwright-cli --session=myapp \
  open https://myapp.com
$ playwright-cli --session=myapp snapshot

# Or use env variable:
PLAYWRIGHT_CLI_SESSION=myapp claude .
```

SUPPORTED AGENTS

Claude Code | Copilot CLI
Any terminal-based AI agent

TEST RUNNER@playwright/test

WHAT IT IS

The built-in test framework for writing and running end-to-end tests with full assertions, reporters, tracing, and CI/CD integration.

INSTALL

```
npm init playwright@latest
# --- or ---
npm install -D @playwright/test
npx playwright install
```

ESSENTIAL COMMANDS

```
npx playwright test
npx playwright test --headed
npx playwright test --ui
npx playwright test --debug
npx playwright codegen <url>
npx playwright show-report
npx playwright show-trace trace.zip
```

USEFUL FLAGS

```
--project=chromium|firefox|webkit
--grep='login'      (filter by title)
--retries=3         (flaky test retry)
--reporter=html     (generate report)
--workers=4         (parallelism)
```

SUPPORTED ENVIRONMENTS

CI/CD | GitHub Actions | Docker
Local dev | Azure App Testing

ASPECT	MCP SERVER	CLI SKILLS	TEST RUNNER
What is it?	A server exposing browser tools to AI agents via JSON-RPC protocol. The agent sends tool calls, the server drives the browser.	A command-line tool that lets coding agents run browser actions as simple shell commands, one at a time.	A test framework that runs your .spec.ts files with assertions, reporters, and built-in browser automation API.
Who uses it?	AI agents: Claude Desktop, Cursor, VS Code Copilot, Windsurf, Goose, or any MCP-compatible client.	Terminal-based coding agents: Claude Code, Copilot CLI, or any shell-based AI assistant.	Human developers and QA engineers writing automated end-to-end tests. Also CI/CD pipelines.
How it sees the page	Structured accessibility snapshots — a tree of all elements with roles, names, and reference IDs. No pixels needed.	Same accessibility snapshots via the 'snapshot' command. Elements referenced by compact IDs like e12, e35.	Full Playwright API in code: locators, getByRole, getByText, CSS selectors. Pixel-level control via page object.
Token cost for AI	HIGH: Loads full tool schemas + verbose accessibility trees into the context window. Can consume 10K+ tokens.	LOW: Short commands, minimal output. Designed to save tokens when the agent also needs code + tests in context.	ZERO: Runs outside the AI context. It is code executed by Node.js, not a tool for AI agents.
State across calls	Browser stays open within a single MCP session. Cookies persist in a dedicated user profile on disk.	Named sessions keep cookies and storage alive across many CLI invocations. Use --session=name to isolate projects.	Each test gets a fresh browser context by default. Use storageState to share authentication across tests.
When to choose it	Exploratory testing, self-healing tests, autonomous browser agents, long-running workflows, interactive debugging.	High-throughput coding agents that need browser + code + tests all in one tight context window. CI scripting.	Regression suites, CI/CD pipelines, visual testing, performance testing, formal reporting with traces.

QUICK DECISION GUIDE:

Need AI to explore a website? Use MCP. | Coding agent needs the browser? Use CLI. | Running test suites? Use Test Runner.

All three approaches use the same Playwright engine underneath. They can be combined: MCP for exploration, CLI for coding workflows, Test Runner for CI/CD execution.

MCP SERVER

AI agent calls these MCP tools:

```
● ● ●
# 1. Navigate to the Todo app
browser_navigate({
  url: "https://demo.playwright.dev/todomvc/"
})

# 2. Take an accessibility snapshot
browser_snapshot()
# Returns element refs: e12, e35, etc.

# 3. Type a todo item
browser_type({
  ref: "e12", text: "Buy groceries"
})

# 4. Submit with Enter key
browser_press_key({ key: "Enter" })

# 5. Type another item
browser_type({
  ref: "e12", text: "Water flowers"
})
browser_press_key({ key: "Enter" })

# 6. Verify – snapshot again
browser_snapshot()

# 7. Generate a test from this session
browser_generate_playwright_test()
# Outputs a .spec.ts file automatically
```

CLI SKILLS

Coding agent runs these commands:

```
● ● ●
# 1. Open with a named session
$ playwright-cli --session=todo \
  open https://demo.playwright.dev/todomvc/ \
  --headed

# 2. Get element references
$ playwright-cli --session=todo snapshot

# 3. Add first todo
$ playwright-cli --session=todo type "Buy groceries"
$ playwright-cli --session=todo press Enter

# 4. Add second todo
$ playwright-cli --session=todo type "Water flowers"
$ playwright-cli --session=todo press Enter

# 5. Mark first as complete
$ playwright-cli --session=todo check e21

# 6. Take a screenshot for proof
$ playwright-cli --session=todo screenshot

# 7. View all network requests
$ playwright-cli --session=todo network

# Session stays alive for more commands
# Cookies persist between calls
```

TEST RUNNER

Developer writes this spec file:

```
● ● ●
import { test, expect } from '@playwright/test';

test('add and complete todos', async ({ page }) => {

  // Navigate to the app
  await page.goto(
    'https://demo.playwright.dev/todomvc/'
  );

  // Add first todo
  const input = page.getByPlaceholder(
    'What needs to be done?'
  );
  await input.fill('Buy groceries');
  await input.press('Enter');

  // Add second todo
  await input.fill('Water flowers');
  await input.press('Enter');

  // Assert both exist
  const items = page.getByTestId('todo-title');
  await expect(items).toHaveCount(2);

  // Complete the first one
  await items.first().check();
});
```

TO RUN THE TEST: `npx playwright test todo.spec.ts --headed --reporter=html`**VIEW RESULTS:** `npx playwright show-report` | `npx playwright show-trace trace.zip`